Robots as Tokens: Unified Diffusion Transformer for Coordinated Multi-Robot Trajectory Generation

Anonymous Author(s)
Affiliation
Address
email

Abstract: The success of generative models in language, image, and video generation has inspired extensive applications to generative robot planning. However, most existing works either focus on single-robot planning, or generate multi-robot trajectories in a sequential manner with iterative post-processing to resolve inter-robot conflicts. In this work, we investigate whether coordinated multi-robot trajectories, as a special spatiotemporal distribution, can be learned and generated with a generative model in a feed-forward manner. We propose **Ro**bots as To**ken**s (Roken), a unified diffusion transformer that directly generates multi-robot trajectories that satisfy both (individual) safety and (global) connectivity constraints. The core design of Roken is to represent each robot as a discrete token, allowing them to naturally interact with each other through self-attention, and cross-attend to map tokens for environment layouts. We further introduce several auxiliary tasks based on Bayes' theorem, specifically local occupancy reconstruction and long-horizon waypoint prediction, to provide multi-scale spatial-temporal supervision for efficient learning of the conditional distribution. In training, Roken absorbs diverse expert trajectories from different team sizes. During inference, Roken behaves as a versatile multi-robot planner that can handle single-robot planning, coordinated multi-robot trajectory generation, and conditional trajectory generation by fixing some robot tokens as conditions. Experiments in diverse cluttered environments show that Roken can generate coordinated multi-robot trajectories to perform connectivity-constrained goal navigation tasks with high success rates, outperforming the baseline method used to generate the training dataset. Roken also demonstrates good scalability after training with mixed team sizes, and shows generalization to unseen or partially observed environments, verifying its potential as a unified multi-robot planner that can learn from diverse data and perform versatile tasks.

Keywords: Multi-Robot Planning, Diffusion Models, Trajectory Generation

1 Introduction

Multi-robot systems have broad applications in search and rescue [1, 2], autonomous exploration [3, 4], and environmental monitoring [5]. To ensure effective coordination, it is essential for the robots to maintain an underlying connected communication network for real-time information sharing and decision-making. However, such connectivity constraints usually conflict with robots' navigation tasks, as the communication links can be easily broken when robots navigate in obstacle-cluttered environments. In such a case, it is challenging to plan coordinated trajectories that not only achieve navigation tasks but also satisfy both safety and connectivity maintenance constraints.

Existing methods consider either continuous-time or discrete-time connectivity maintenance. The former are usually based on potential fields or optimization-based methods [6, 7], while the latter rely on discrete candidate sampling and combinatorial evaluation [8, 9]. However, in cluttered

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

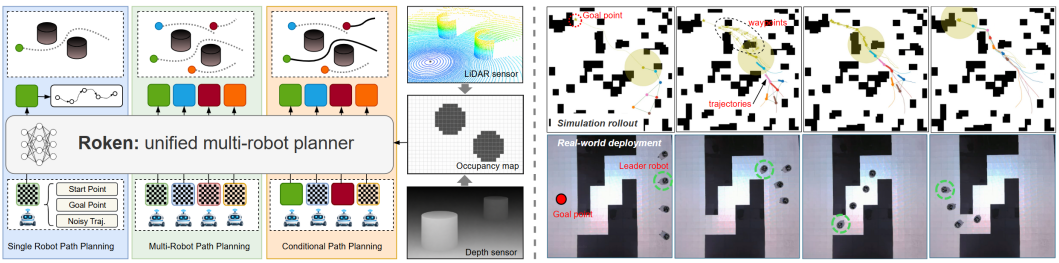


Figure 1: (Left) Roken supports three planning modes with the same robot-token diffusion model: single-robot planning, coordinated multi-robot planning, and conditional planning with fixed robot tokens. (Right) Deployment of Roken in a cluttered simulation environment with seven robots, and in a real-world maze environment with four robots, all with a single Roken model.

environments, the gradient field-based methods can easily get stuck in local minima areas due to confliction of multiple components, including collision avoidance, goal navigation, and communication maintenance. The discrete approaches suffer from the curse of dimensionality for candidates evlauation, and often require downstream repair to generate collision-free trajectories [9, 10].

Recently, generative models have shown strong capability in modeling complex distributions like language, images and videos [11, 12, 13, 14], and they have been applied to generative trajectory planning in robot navigation and manipulation tasks [15, 16, 17, 18, 19]. However, most diffusion planning works have focused on single robot planning, or fixed-environment trajectory generation [15]. The individually generated trajectory either needs to be conditioned on existing robot plans, or requires further iterative refinement to satisfy multi-robot constraints [20, 21, 22]. This motivates us to ask the question: can we learn a generative model that directly generates coordinated multi-robot trajectories in a feed-forward manner, and scales to varying numbers of robots?

In this work, we propose Robots as Tokens (Roken), a unified generative model based on diffusion transformer (DiT) [23, 24] that treats coordinated multi-robot trajectories as a special spatiotemporal distribution, and learns the map-conditioned trajectory distribution from expert data. The key design of Roken is to represent each robot as a token input to the DiT model, where the transformer structure naturally allows robot tokens to interact with each other through self-attention, and attend to environment map tokens for spatial awareness. In training, Roken absorbs expert trajectories with different team sizes as different numbers of robot tokens, and supports conditional training by treating some robot tokens as known conditions. During inference, Roken behaves as a versatile multi-robot planner that can handle single-robot planning, coordinated multi-robot planning, and conditional planning by simply changing the number of robot tokens and their trajectory masks, without any change to the model architecture or parameters. This mimics the versatility of human planners, who can not only play Ludo with only one role, but also play chess with many different roles and opponents' conditions. The contributions of this paper are:

- We propose Roken, a unified diffusion transformer that encodes each robot as a token, learns and generates communication-constrained coordinated trajectories of varying numbers of robots in a feed-forward manner, without sequential individual planning or iterative post-processing.
- We design multi-scale spatial-temporal self-supervision through auxiliary tasks for conditional distribution learning, enhancing the condition injection for the model's map awareness, long-range planning, and collision avoidance behaviors.
- We conduct extensive experiments to verify the capabilities, scalability, and generalizability of Roken in diverse complex maps, and demonstrate its deployment in real-world navigation tasks.

2 Related Work

2.1 Multi-Robot Planning and Connectivity Maintenance

Multi-robot navigation under communication constraints requires balancing navigation tasks with global connectivity maintenance. Previous gradient-based methods usually encode communication

and collision constraints as potential functions in a connectivity controller [6, 7, 25]. While mathematically elegant, they are inherently reactive and suffer from severe local minima in cluttered environments where navigation and connectivity objectives conflict. Alternatively, communication-aware coordination can be formulated as a combinatorial planning problem by searching for optimal future connected topologies from discrete candidates [9]. However, this discretization introduces coarse motions, often requiring complex downstream smoothing or trajectory repair to generate executable and collision-free paths. Recent works have applied reinforcement learning (RL) to connectivity-aware navigation. The decision space is usually simplified to discrete waypoints selection [26] or candidate topologies scoring [27], leaving continuous execution to external low-level controllers. Moreover, the coordinated behaviors are usually learned through explicit gradient guidance from constarints formulations, which makes it difficult to scale their framework to other collaborative tasks. These limitations motivate us to design a pure data-driven learning framework that directly learns the conditional trajectory distribution from expert demonstrations, without explicit connectivity constraints modeling or gradient injection, making the framework more transferable across different scenarios.

2.2 Diffusion Models for Trajectory Generation

Diffusion models, known to be excellent at modeling complex distributions, have revolutionized image synthesis [11, 12], video generation [13, 14], and robot planning [15, 16, 17, 18]. While many existing methods focus on single-agent control, diffusion models have recently been applied to generative multi-robot motion planning [20, 21, 22]. Despite their significant strides, existing diffusion-based planners predominantly rely on external mechanisms to ensure trajectory feasibility. They typically depend on computationally expensive iterative projections to repair constraint violations [21, 22], or require coupling with discrete search solvers for combinatorial guidance [20, 22]. Different from existing methods, this work presents a unified model that learns coordinated multi-robot trajectories as a joint spatiotemporal distribution, and directly generates multi-robot trajectories in a feed-forward manner, without any iterative refinement, which is more efficient and end-to-end learnable. We provide the global occupancy map as a common conditioning for trajectory generation instead of ego-centric observations as in single-robot settings, which requires each robot token to actively query the relevant information but improves generalizability across different sensors.

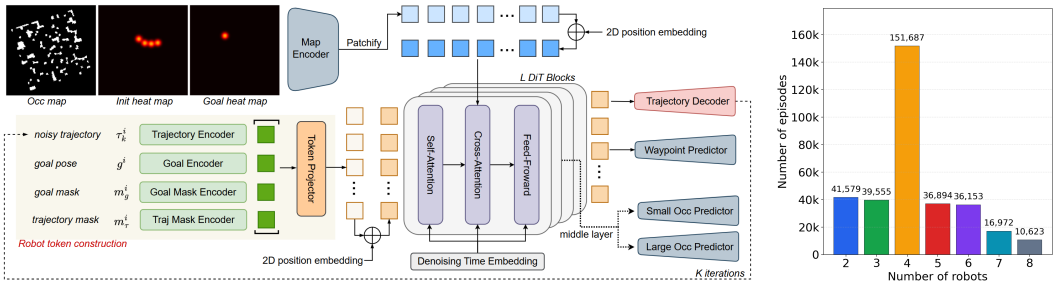
3 Preliminaries and Problem Formulation

3.1 Communication Connectivity

Let $\mathcal{N} = \{1, \dots, N\}$ be a team of robots that navigate in an environment $\mathcal{W}$ with obstacles. Two robots are considered to be in communication if they are within a certain distance R of each other, i.e., $\|p_t^i - p_t^j\|_2 \leq R$, where p_t^i denotes the position of robot i at time t. The communication graph at time t is defined as $\mathcal{G}_t = \langle \mathcal{V}_t, \mathcal{E}_t \rangle$, where $\mathcal{V}_t = \{p_t^i \mid i \in \mathcal{N}\}$ is the set of robots, and $\mathcal{E}_t = \{(i,j) \mid \|p_t^i - p_t^j\|_2 \leq R\}$ is the set of connected edges. Given the communication graph $\mathcal{G}_t$, its graph Laplacian matrix can be obtained as $\mathcal{L}_{\mathcal{G}_t} = D - A$, where A is the adjacency matrix of $\mathcal{G}_t$ and D is the diagonal degree matrix. The graph $\mathcal{G}_t$ is connected if there is a path between any two vertices in the graph, or equivalently, the second-smallest eigenvalue λ_2 of $\mathcal{L}_{\mathcal{G}_t}$ is positive, i.e., $\lambda_2(\mathcal{L}_{\mathcal{G}_t}) > 0$. Otherwise, $\lambda_2(\mathcal{L}_{\mathcal{G}_t}) = 0$ if $\mathcal{G}_t$ is disconnected [28]. The gradient of λ_2 with respect to robot positions can be derived to provide directions for increasing connectivity, which is commonly used in gradient-based connectivity maintenance methods [6, 7, 25].

3.2 Problem Formulation

The environment is represented as a 2D occupancy map $\mathcal{H}_{\text{map}} \in \{0,1\}^{H \times W}$, where occupied cells correspond to obstacles and others represent free space. There are N robots navigating in the environment, where each robot i has an initial position $p_0^i \in \mathbb{R}^2$. One leader robot will be given



(a) Network Architecture of Roken. (b) Dataset distribution.

Figure 2: Overview of the proposed Roken framework and the training dataset distribution across team sizes.

a goal position $g \in \mathbb{R}^2$. The trajectory of a robot i is represented as $\tau^i = \{p_1^i, \ldots, p_T^i\}, \quad i \in \mathcal{N}$, where p_t^i is the position of robot i at time t. The team trajectories are represented as $\mathcal{T} = \{\tau^i\}_{i=1}^N$. The coordinated navigation task requires robots to satisfy the following constraints: (1) **Goal-reaching**: the leader robot should reach the goal position within a specified tolerance ϵ_g, i.e., $\|p_T^i - g\|_2 \leq \epsilon_g$ for the robot i assigned with the goal; (2) **Collision avoidance**: all robots should avoid collisions with obstacles and each other, i.e., $d_{\text{obs}}(p_t^i) \geq d_{\text{obs}}^{\min}$ and $\|p_t^i - p_t^j\|_2 \geq d_{\text{coll}}^{\min}$ for all $i \neq j$ and all t, where $d_{\text{obs}}(p)$ is the distance from position p to the nearest obstacle, and $d_{\text{obs}}^{\min}$ and $d_{\text{coll}}^{\min}$ are safety thresholds; (3) **Communication maintenance**: the team should maintain communication connectivity, i.e., $\lambda_2(\mathcal{L}_{\mathcal{G}_t}) > 0$ for all t.

3.3 Dataset Collection

We collect the training dataset $\mathcal{D}$ with diverse team sizes using a classical potential field-based method **Laplacian** adopted from [7] based on the graph Laplacian. The dataset distribution is shown in Fig. 2b. To provide diverse training environments, we use a 2D occupancy map dataset from the 2d-path-planning-dataset[1], and select 5000 maps from the dataset for training and 1000 maps for evaluation. During dataset collection, the **Laplacian** method first randomly samples start and goal positions, and lets the leader robot follow an optimal A* path while others maintain connectivity. The team only maintains necessary connections with topology optimization following [8]. We smooth the expert trajectories to mitigate the discontinuity, and augment the dataset with random rotation to increase its diversity. During training, we randomly sample a fixed-length multi-robot action chunk and a goal position along the future trajectory of the leader robot from an episode. Note that we abuse the notation $\mathcal{T}$ to represent either the full trajectory or the relative action chunk for notation convenience.

4 Methodology

4.1 Model Structure

We first give an overview of the proposed neural network architecture, called Roken, as shown in Fig. 2(a). Roken is an end-to-end diffusion transformer, which takes multiple *robot tokens* as inputs, and passes them through multiple transformer decoder layers while cross-attending to *map tokens* to extract occupancy information, and outputs latent vectors which are then decoded into clean trajectories. The overall training objective is to maximize the conditional likelihood of feasible coordinated trajectories:

$$\max_{\theta} \mathbb{E}_{\mathcal{T}, \mathcal{H}_{\text{map}}, \mathcal{P}_0, g \sim p_{\text{data}}} \left[\log p_\theta(\mathcal{T} | \mathcal{H}_{\text{map}}, \mathcal{P}_0, g) \right]. \tag{1}$$

Robot Tokens: We introduce the novel concept of robot tokens in this work for scalable multi-robot trajectory generation. Each robot token is a latent vector that encapsulates the information of a single

[1] https://www.kaggle.com/datasets/dcaffo/2dpathplanningdataset

robot, including its noisy future action chunk and goal information, defined as

$$z_0^i = \phi_{\text{tok}}\left(\text{CAT}\left[\phi_\tau(\tau_k^i), \phi_g(g^i - p_0^i), \phi_{mg}(m_g^i), \phi_{m\tau}(m_\tau^i)\right]\right) + \text{PE}(p_0^i), \tag{2}$$

where ϕ_τ, ϕ_g, ϕ_{mg}, and $\phi_{m\tau}$ are embedding layers for the noisy action chunk τ_k^i at the k-th denoisingtime step, relative goal $g^i - p_0^i$, goal mask $m_g^i \in \{0, 1\}$, and trajectory mask $m_\tau^i \in \{0, 1\}$, respectively. The goal mask indicates the leader robot; and the trajectory mask indicates whether the action chunk is generated from noise or kept fixed as a condition for conditional generation. The linear projection ϕ_{tok} maps the concatenated embeddings into the robot token space, and adds sinusoidal positional embeddings $\text{PE}(p_0^i)$. All input robot tokens are denoted as $Z_0 = \{z_0^i\}_{i=1}^N$.

With robot tokens, our unified DiT model Roken enjoys three advantages: (1) it can accept varying numbers of robots by simply changing the number of robot tokens; (2) it is permutation-invariant to the order of robot tokens, which is natural for multi-robot systems; and (3) it allows flexible training with diverse datasets containing different team sizes, as well as conditional training by treating some robot tokens as known conditions and others to be generated from noise.

Map tokens: To help the ResNet-based map encoder learn relevant representations for robots rather than a general map compression, we extend the map tensor with two additional channels: a robot-start heatmap and a goal heatmap. A Gaussian kernel is placed at each robot's start position and at the goal position in their respective channels, providing spatial anchors for the map encoder to better compress the map information. After the map encoder, the downsampled map is divided into patch tokens with positional embeddings from the patch center coordinates. Robot tokens attend to these map tokens through transformer decoder cross-attention, with $Z_{\ell+1} = \text{DecoderLayer}_\ell(Z_\ell, M, e_k)$, where $Z_\ell = \{z_\ell^i\}_{i=1}^N$ are N input latent embeddings at the ℓ-th layer, $M = \{h^1, \ldots, h^m\}$ are m map patch tokens, and e_k is the diffusion timestep embedding. The denoising time step embedding is injected into the network with AdaLN-Zero [23] for stable diffusion transformer training.

Auxiliary Heads: While it is straightforward to use diffusion-based imitation learning to train the trajectory decoder head, we found that this alone is difficult for effective learning of the map-conditioned trajectory distribution. The main reason is that the relative goal position $g - p_0^i$ in robot token z_0^i provides a strong shortcut for the model to simply learn to move towards the goal, without querying the map information. To show that, we can rewrite the learning objective in Eq. (1) following the Bayesian formula as follows:

$$\log p(\mathcal{T} \mid \mathcal{H}_{\text{map}}, \mathcal{P}_0, g) = \log p(\mathcal{T} \mid \mathcal{P}_0, g) + \log p(\mathcal{H}_{\text{map}} \mid \mathcal{T}, \mathcal{P}_0, g) - \log p(\mathcal{H}_{\text{map}} \mid \mathcal{P}_0, g). \tag{3}$$

Due to the existence of the training shortcut, the model actually learns $p(\mathcal{T} \mid \mathcal{P}_0, g)$, rather than the desired conditional distribution $p(\mathcal{T} \mid \mathcal{H}_{\text{map}}, \mathcal{P}_0, g)$. Therefore, we introduce the *local occupancy reconstruction* heads to enhance the learning of the second term $p(\mathcal{H}_{\text{map}} \mid \mathcal{T}, \mathcal{P}_0, g)$, thereby supplementing the learning of the target conditional distribution in Eq. (3). Note that the term $\log p(\mathcal{H}_{\text{map}} \mid \mathcal{P}_0, g)$ is unrelated to $\mathcal{T}$ and can be treated as a constant. The occupancy reconstruction head asks each middle-layer latent vector to decode two local occupancy maps with both original and large patch scales, centered around the corresponding robot position. This encourages the robot token to localize itself within the map, aggregate surrounding discrete map patches, and understand both fine-grained and large-scale obstacle layouts for trajectory planning.

We also introduce another auxiliary *waypoint prediction* head to enhance the long-range planning capability of the model, which predicts a few sparse but farther waypoints along the trajectory. Together, the two auxiliary heads form a multi-scale spatial-temporal supervision for the model. They are only used during training and are disabled during inference.

4.2 Training Objectives

The training objective in Eq. (1) is optimized through a denoising score-matching loss with several auxiliary losses. The overall loss function is defined as:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{traj}} + \lambda_2 \mathcal{L}_{\text{wp}} + \lambda_3 \mathcal{L}_{\text{occ}} + \lambda_4 \mathcal{L}_{\text{sdf}}, \tag{4}$$

where $\mathcal{L}_{\text{traj}} = \mathbb{E}\left[\|\epsilon - \epsilon_\theta(\mathcal{T}_k, \mathcal{P}_0, g, \mathcal{H}_{\text{map}}, k)\|_2^2\right]$ is the trajectory denoising loss, $\epsilon \sim \mathcal{N}(0, I)$ is the added noise at the k-th diffusion step, and $\epsilon_\theta(\cdot)$ is the added predicted noise from the model. $\mathcal{T}_k = \sqrt{\bar{\alpha}_k}\mathcal{T}_0 + \sqrt{1-\bar{\alpha}_k}\epsilon$ is the noisy action chunk perturbed from $\mathcal{T}_0$, where $\bar{\alpha}_k$ is the cumulative product of the noise schedule. The auxiliary losses include three terms: $\mathcal{L}_{\text{wp}}$ is the waypoint prediction MSE loss; $\mathcal{L}_{\text{occ}}$ is the local occupancy reconstruction loss, for both small and downsampled large patches with binary cross entropy loss; and $\mathcal{L}_{\text{sdf}}$ is the SDF loss to enhance the collision avoidance performance, defined as $\mathcal{L}_{\text{sdf}} = \left[\text{ReLU}\left(d_{\text{thres}} - d_{\text{sdf}}\right)\right]^2$, where d_{sdf} is the signed distance from the generated trajectory or discrete waypoints to the nearest obstacle, and d_{thres} is a safety threshold.

We train Roken following a pre-training and post-training paradigm. The pre-training stage trains the model with the loss function $\mathcal{L}$ on expert datasets, which alone has shown strong performance in generating collision-free and communication-maintained trajectories. However, the model occasionally falls short in escaping from the local minima areas in cluttered environments. Therefore, we apply the post-training stage with trajectory-level reinforcement learning to further improve the safety and goal-reaching performance of the model. Specifically, we adopt the group relative policy optimization (GRPO) algorithm [29] for critic-free policy optimization. During policy rollout, we sample an initial configuration from a random time step of the expert trajectory, and perform parallel generation of multi-robot trajectories, covering both safe and unsafe candidates. The rewards of each candidate are then scored within the group. Note that we inject the privilege information about the geodesic distance to the reward function to enhance the global planning performance of the model, rather than only focusing on the short-term best candidates.

5 Experimental Results

We evaluate Roken around capability, generalizability, and scalability. Specifically, we evaluate (1) whether a unified DiT model can learn to generate coordinated multi-robot trajectories that satisfy safety and communication constraints; (2) what is the effect of auxiliary tasks compared to only using the denoising objectives; (3) whether Roken can scale with varying team sizes and generalize to unseen and out-of-distribution maps. Our model is implemented in PyTorch and trained on an NVIDIA A6000 GPU. The model contains about 11.9 million parameters, with 8 transformer decoder layers and a hidden dimension of 512. We pad the robot tokens to a maximum of 10. The occupancy map is padded to a size of 140×140, downsampled by a ResNet [30] encoder and then divided into discrete 10×10 patches. The AdamW optimizer [31] is used with a learning rate of 5e-4.

Baseline methods: We compare Roken with three baselines that represent different paradigms for multi-robot connectivity maintenance. **Laplacian** [8] is a continuous gradient field-based approach based on graph-Laplacian for connectivity maintenance. **SGG** [9] formulates multi-robot coordination as a discrete topology optimization problem with continuous refinement. **GNN** [27] trains a Graph Neural Network (GNN)-based model to score candidate future topology graphs of robots, and the next position for each robot is selected sequentially based on the scores.

Evaluation metrics: We evaluate the performance of different methods on 1000 unseen environments with varying layouts and initial configurations. Based on the problem formulation, we define five metrics to evaluate the performance of different methods: obstacle collision ratio (Obs-Collision), inter-robot collision ratio (Inter-Collision), team connectivity ratio, goal reach ratio (Reach), and distance efficiency (Length). The distance efficiency is measured by the average trajectory length of all robots in successful episodes, normalized by the discrete A* path length. We summarize the performance with Full Success, where a successful episode requires the team to reach the target while *always* satisfying both safety and communication constraints. Therefore, the full success ratio is a stringent and comprehensive metric, along with the hard collision avoidance metrics. The connectivity ratio or reach ratio is less reliable because the model can generate invalid trajectories that violate safety constraints but still get high ratios.

Table 1: Evaluation results on unseen environments with four robots.

Model	$\mathcal{L}_{\text{traj}}$	$\mathcal{L}_{\text{occ}}$	$\mathcal{L}_{\text{wp}}$	$\mathcal{L}_{\text{sdf}}$	Full success ↑	Obs-Colli. ↓	Inter-Colli. ↓	Connect. ↑	Reach ↑	Length ↓
SGG [9]		–			0.479	**0.000**	0.028	0.505	0.976	**0.492**
GNN [27]		–			0.327	0.329	**0.000**	0.414	0.999	2.174
Laplacian [8]		–			0.741	0.098	0.053	0.949	0.784	1.178
Roken$_{\text{four}}$	✓	✓	✓	✓	0.765	0.145	0.081	0.873	0.923	0.623
Roken$_{\text{mixed}}$	✓	✓	✓	✓	0.779	0.112	0.098	0.899	0.915	0.629
Roken$_{\text{RL}}$	✓	✓	✓	✓	**0.790**	0.127	0.089	0.920	0.947	0.622
	✓	×	×	×	0.262	0.707	0.110	**0.997**	**1.000**	0.523
Roken$_{\text{four}}$	✓	✓	×	×	0.724	0.179	0.088	0.844	0.894	0.631
Ablations	✓	✓	✓	×	0.698	0.225	0.101	0.874	0.910	0.619
	✓	✓	✓	✓	0.719	0.164	0.122	0.865	0.909	0.628

Note: All Roken$_{\text{four}}$ ablation models are trained for 200 epochs on the same four-robot datasets. All methods are evaluated on 1000 unseen environments with five independent runs, which in total involve 5000 evaluation episodes. Bold: best performance; Underline: second-best performance.

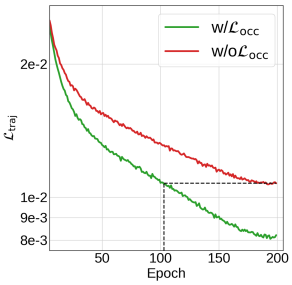


Figure 3: Training loss of $\mathcal{L}_{\text{traj}}$ w/ and w/o $\mathcal{L}_{\text{occ}}$ in Roken$_{\text{four}}$ ablations.

5.1 Performance Evaluation on Distribution Learning

Comparison with baseline methods: We first compare the performance of Roken with the baseline methods on the unseen test environments, as shown in Table 1. It is worth noting that SGG, GNN, and Laplacian use the global optimal A* path for the leader robot, while Roken does not. Despite that, both of our models Roken$_{\text{four}}$ (trained on four-robot dataset) and Roken$_{\text{mixed}}$ (trained on mixed dataset with two to eight robots) outperform the baseline methods. Roken$_{\text{mixed}}$ performs slightly better than Roken$_{\text{four}}$ in terms of full success ratio, which indicates that absorbing training data with diverse team sizes can help to learn more generalized coordination behaviors. The post-trained model Roken$_{\text{RL}}$ further improves the full success rate to 0.790, with better reach ratio and connectivity ratio, demonstrating the successful enhancement of the global planning performance of the model. In contrast, although the compared baseline methods follow safety constraints better than Roken, they fail to maintain connectivity and reach the goal at the same time, leading to much lower full success ratios. The Laplacian method achieves a much higher full success rate of 0.741, but it suffers from a lower reach ratio because robots are frequently stuck in conflicting potential fields, which also results in longer zig-zag trajectories.

Ablation studies: We train a series of Roken$_{\text{four}}$ ablation models to investigate the contribution of each auxiliary loss. As shown in Tab. 1, with only $\mathcal{L}_{\text{traj}}$, a vanilla DiT model is difficult to learn the map-conditioned trajectory distribution, with the highest obstacle collision ratio. The largest improvement happens after adding $\mathcal{L}_{\text{occ}}$, which significantly improves the full success ratio from 0.262 to 0.724. Besides that, $\mathcal{L}_{\text{occ}}$ also helps to double the training speed with a much lower $\mathcal{L}_{\text{traj}}$, as shown in Fig. 3. This supports our analysis on the conditional distribution learning with auxiliary occupancy reconstruction loss. The $\mathcal{L}_{\text{wp}}$ improves the reach ratio by avoiding local minima areas through long-term waypoint prediction, but it competes with the trajectory tasks and therefore leads to a slight drop of the full success rate. $\mathcal{L}_{\text{sdf}}$ additionally helps to improve the collision avoidance performance, achieving the lowest obstacle collision ratio of 0.164 in all ablation models.

5.2 Scalability Evaluation with Varying Team Sizes

Roken is designed to be scalable to different team sizes, in both training and inference. We compare three variants of Roken trained on different recipes of datasets: Roken$_{\text{four}}$, Roken$^{\text{masked}}_{\text{four}}$, and Roken$_{\text{mixed}}$, where Roken$^{\text{masked}}_{\text{four}}$ randomly masks robot tokens during training for conditional generation. The results are reported in Fig. 4. Overall, Roken$_{\text{mixed}}$ shows the best scalability performance across different team sizes, with a more stable full-success ratio as the team size increases, even though the training data distribution is heavily imbalanced towards the four-robot case, as shown in Fig. 2b. This verifies our assumption that by the robot-token design, Roken learns generalizable coordination behaviors across diverse team sizes. All three variants show high full success ratios for single-robot planning (although we never train them on single-robot datasets), and high goal-reaching ratios for all team sizes, indicating that the model learns strong goal-oriented behaviors especially for the leader robot. Roken$_{\text{four}}$ shows some generalization to three or five robots, but fails to capture the coordinated movements as team size increases, leading to frequent inter-robot colli-

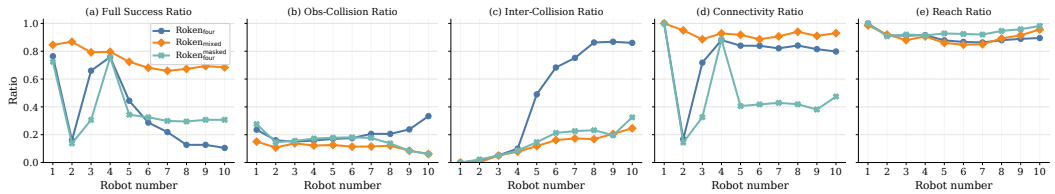


Figure 4: Scalability evaluation from one to ten robots for three variants of Roken. We rolled out 500 episodes in 500 unseen maps for each team size and model variant.

sions. With clustered conditional generation, $\text{Roken}_{\text{four}}^{\text{masked}}$ can achieve better success ratio for larger team sizes, with good collision avoidance performance. However, it also introduces disturbances for the model to learn the connectivity maintenance behaviors, as indicated by the significant drop of connectivity ratio with larger team sizes.

5.3 Generalizability and Real-world Deployment

This section evaluates the generalizability of Roken from global maps to local and out-of-distribution maps. We conduct zero-shot evaluation of Roken_{RL} by inputting only partially observed maps around robots, which is more realistic for real-world applications. The full success ratios in 500 episodes with unseen maps are reported in Table 2, with different sensing ranges defined in the pixel space. When the sensing range is large, the model maintains a similar success ratio as the global map input, verifying its map query capability. However, as the sensing range decreases to be very small, the generated trajectories are more likely to get stuck before realizing the map structure, which indicates the limitation of our model in escaping from local minima areas with only short-term observation. We also deploy Roken on a real-world multi-robot system with various numbers of omnidirectional robots, as shown in Fig. 1. We manually design cluttered environments that are different from the training maps. Despite that, Roken can still generate coordinated trajectories that maintain connectivity and avoid obstacles, which verifies the real-world applicability of the model.

Table 2: Success ratios under different sensing ranges.

Sensing range	Full success
Full map	0.836
20	0.832
10	0.782
5	0.600

6 Conclusion

This work studies the connectivity-constrained multi-robot trajectory generation problem. We proposed Roken, a unified DiT model that represents robots as tokens, learns and generates coordinated multi-robot trajectories in a feed-forward manner. We propose multi-scale spatial-temporal self-supervision to enhance the conditional distribution learning of the model, and further improve the safety and long-horizon planning capabilities of the model through trajectory-level reinforcement learning. Roken demonstrates strong capability in generating coordinated trajectories that satisfy safety and communication constraints, outperforming baseline methods used to collect training datasets. After training on a mixed dataset with varying team sizes, Roken shows good scalability to different team sizes with stable success rates. We also demonstrate the model's ability to generalize to partially known and out-of-distribution maps, with successful real-world deployment.

7 Limitations

As mentioned in Sec. 5.3, Roken falls short in generating the escaping trajectories once it gets stuck in local minima areas, mainly because the training data lacks sufficient examples of such scenarios. Moreover, the current local occupancy reconstruction tasks only require the model to reconstruct a pre-defined small and large patches, which limits the robot tokens to acquire the global horizon. Finally, despite the improved obstacle avoidance performance of the model, a safety shield may still be needed to ensure the safety in real-world applications.

References

[1] Y. Tian, K. Liu, K. Ok, L. Tran, D. Allen, N. Roy, and J. P. How. Search and rescue under the forest canopy using multiple uavs. *The International Journal of Robotics Research*, 39(10-11): 1201–1221, 2020.

[2] Y. Wang, Y. Cao, J. Chiun, S. Koley, M. Pham, and G. A. Sartoretti. Viper: Visibility-based pursuit-evasion via reinforcement learning. In *8th Annual Conference on Robot Learning*, 2024.

[3] R. Bai, H. Guo, W.-Y. Yau, and L. Xie. Graph-based slam-aware exploration with prior topo-metric information. *IEEE Robotics and Automation Letters*, 9(9):7597–7604, 2024.

[4] J. Chiun, S. Zhang, Y. Wang, Y. Cao, and G. Sartoretti. Marvel: Multi-agent reinforcement learning for constrained field-of-view multi-robot exploration in large-scale environments. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11392–11398. IEEE, 2025.

[5] X. Xu, M. Cao, S. Yuan, T. H. Nguyen, T.-M. Nguyen, and L. Xie. A cost-effective cooperative exploration and inspection strategy for heterogeneous aerial system. In *2024 IEEE 18th International Conference on Control & Automation (ICCA)*, pages 673–678. IEEE, 2024.

[6] P. Robuffo Giordano, A. Franchi, C. Secchi, and H. H. Bülthoff. A passivity-based decentralized strategy for generalized connectivity maintenance. *The International Journal of Robotics Research*, 32(3):299–323, 2013.

[7] T. Nestmeyer, P. Robuffo Giordano, H. H. Bülthoff, and A. Franchi. Decentralized simultaneous multi-target exploration using a connected network of multiple robots. *Autonomous robots*, 41:989–1011, 2017.

[8] R. Bai, S. Yuan, X. Xu, X. Ji, X. Li, H. Guo, W.-Y. Yau, and L. Xie. Line-of-sight-constrained multi-robot mapless navigation via polygonal visible regions, 2026. URL https://arxiv.org/abs/2603.26314.

[9] G. Shi, I. E. Rabban, L. Zhou, and P. Tokekar. Communication-aware multi-robot coordination with submodular maximization. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 8955–8961, 2021. doi:10.1109/ICRA48506.2021.9561041.

[10] M. Ishat-E-Rabban, G. Shi, G. Bonner, and P. Tokekar. Fast k-connectivity restoration in multi-robot systems for robust communication maintenance. In *International Symposium on Distributed Autonomous Robotic Systems*, pages 553–568. Springer, 2024.

[11] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[12] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.

[13] A. Blattmann, T. Dockhorn, S. Kulal, D. Mendelevitch, M. Kilian, D. Lorenz, Y. Levi, Z. English, V. Voleti, A. Letts, et al. Stable video diffusion: Scaling latent video diffusion models to large datasets. *arXiv preprint arXiv:2311.15127*, 2023.

[14] T. Wan, A. Wang, B. Ai, B. Wen, C. Mao, C.-W. Xie, D. Chen, F. Yu, H. Zhao, J. Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.

[15] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning*, pages 9902–9915, 2022.

[16] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.

[17] A. Sridhar, D. Shah, C. Glossop, and S. Levine. Nomad: Goal masked diffusion policies for navigation and exploration. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 63–70. IEEE, 2024.

[18] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[19] J. Chen, Y. Cai, Y. Wang, R. Bai, Y. Cao, J. Li, Y. W. Yun, and G. Sartoretti. Imaginav: Scalable embodied navigation via generative visual prediction and inverse dynamics. *arXiv preprint arXiv:2603.13833*, 2026.

[20] Y. Shaoul, I. Mishani, S. Vats, J. Li, and M. Likhachev. Multi-robot motion planning with diffusion models. *arXiv preprint arXiv:2410.03072*, 2024. URL https://arxiv.org/abs/2410.03072.

[21] J. Liang, J. K. Christopher, S. Koenig, and F. Fioretto. Simultaneous multi-robot motion planning with projected diffusion models. *arXiv preprint arXiv:2502.03607*, 2025. URL https://arxiv.org/abs/2502.03607.

[22] J. Liang, S. Koenig, and F. Fioretto. Discrete-guided diffusion for scalable and safe multi-robot motion planning. *arXiv preprint arXiv:2508.20095*, 2025. URL https://arxiv.org/abs/2508.20095.

[23] W. Peebles and S. Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4195–4205, 2023.

[24] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[25] R. Bai, S. Yuan, K. Li, H. Guo, W.-Y. Yau, and L. Xie. Realm: Real-time line-of-sight maintenance in multi-robot navigation with unknown obstacles. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 7363–7369. IEEE, 2025.

[26] M. Li, Y. Jie, Y. Kong, and H. Cheng. Decentralized global connectivity maintenance for multi-robot navigation: A reinforcement learning approach. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 8801–8807, 2022. doi:10.1109/ICRA46639.2022.9812163.

[27] W. Tang, C. Li, J. Wu, and Q. Zhu. Decentralized communication-maintained coordination for multi-robot exploration: Achieving connectivity and adaptability. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3417–3424, 2024. doi:10.1109/IROS58592.2024.10802832.

[28] M. Fiedler. Algebraic connectivity of graphs. *Czechoslovak mathematical journal*, 23(2): 298–305, 1973.

[29] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[30] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[31] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*.

A Hyperparameters

Task specifications:

Table 3: Task constraint parameters used in evaluation.

Parameter	Value	Description
R	15.0	Maximum communication radius between two robots
$d_{\text{coll}}^{\min}$	2.0	Minimum allowed distance between two robots
$d_{\text{obs}}^{\min}$	1.0	Minimum allowed distance from a robot to obstacles
ϵ_g	1.0	Goal-reaching threshold distance

Note: All values are in the pixel space, where 1 pixel corresponds to 1 meter in the real world.

Roken model:

Table 4: Roken model hyperparameters.

Hyperparameter	Value
Number of transformer decoder layers	8
Number of attention heads	8
Hidden dimension	512
Maximum number of robot tokens	10
Occupancy map size (pixels)	140×140
Occupancy patch size (pixels)	10×10
Reconstructed local small patch size (pixels)	7×7
Reconstructed local large patch size (pixels)	7×7
Number of predicted waypoints	5
Waypoints sampling interval (meters)	8
Action chunk prediction horizon	16
Action maximum step size (meters)	0.5
Trajectory sampling interval (steps)	4
Learning rate	5e-4
Batch size	128
λ_1 (trajectory loss weight)	1.0
λ_2 (waypoint loss weight)	1.0
λ_3 (occupancy loss weight)	0.1
λ_4 (SDF loss weight)	1.0
d_{thres} (SDF safety threshold, pixels)	4.0

B Dataset Collection Pipeline

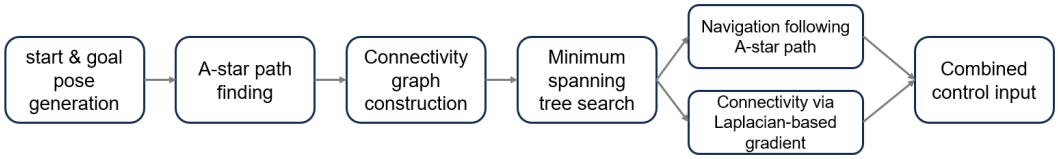


Figure 5: Pipeline for dataset collection.

This section provides detailed description of the dataset collection pipeline, which is based on a classical gradient field-based method adopted from [7] and [8]. The detailed process is illustrated in Fig. 5. Specifically, a communication graph $\mathcal{G}_t$ is first constructed as defined in Sec. 3.1. The edge weight w_{ij} for each edge (i, j) is defined as the product of the potential function values for different inter-robot constraints, as defined in [8]. The difference is that in this paper we do not consider the line-of-sight constraint, and therefore the edge weight is only determined by the communication distance between two robots and the distance to obstacles. Given the weighted communication graph $\mathcal{G}_t$, a minimum spanning tree $\mathcal{T}_\text{G}$ is then searched to determine the minimum topology for connectivity maintenance, where the effort for maintaining the connectivity while navigating to the goal point is minimized. With $\mathcal{T}_\text{G}$ and its related graph Laplacian matrix $\mathbf{L}_{\mathcal{T}_\text{G}}$, the gradient field for maintaining team connectivity is derived as

$$\boldsymbol{u}_i^c = -\frac{\partial V^\lambda(\lambda_2)}{\partial \lambda_2} \cdot \frac{\partial \lambda_2}{\partial p_t^i} = -\frac{\partial V^\lambda(\lambda_2)}{\partial \lambda_2} \cdot \sum_{j \in \mathcal{N}_i} \frac{\partial A_{ij}}{\partial p_t^i} \left(v_{2i} - v_{2j}\right)^2, \tag{5}$$

where $V^\lambda(\lambda_2)$ is a potential function of the second smallest eigenvalue λ_2 of $\mathbf{L}_{\mathcal{T}_\text{G}}$ defined as $V^\lambda(\lambda_2) = \frac{1}{\lambda_2 - \lambda_2^{\min}}$, where $\lambda_2^{\min}$ is the preferred minimum value of λ_2; v_{2i} is the i-th element of the corresponding eigenvector $\boldsymbol{v}_2$; $\mathcal{N}_i$ is the set of neighbors of robot i; and A_{ij} is the edge weight between robot i and j.

To avoid robots getting stuck when navigating to the goal point, we provide the optimal A* path for the leader robot as a reference trajectory (note that Roken does not have such a reference trajectory, and only relies on the map input for end-to-end trajectory planning). The final movement of the leader robot is determined by the combination of the connectivity potential field and the navigation potential field, defined as

$$\boldsymbol{u}_i = \lambda_c \cdot \boldsymbol{u}_i^c + \lambda_r \cdot \boldsymbol{u}_i^n, \tag{6}$$

where $\boldsymbol{u}_i^n$ is the navigation potential field defined as $\boldsymbol{u}_i^n = \frac{g_t - p_t^i}{\|g_t - p_t^i\|}$, where g_t is a reference waypoint along the reference A* trajectory for the leader robot. The two potential fields are normalized and weighted by λ_c and λ_r to determine the final movement of the leader robot. For the remaining robots, their movements are determined by the connectivity potential field only, which coordinates them to follow the leader robot while maintaining connectivity.

Lazy movement: Because of the minimum spanning tree search module, the final robot trajectories exhibit a *lazy movement* behavior, where robots only move when it is necessary to maintain connectivity, and they tend to stay still when possible. This is to avoid unnecessary movements and to make robots more dispersed in the environment with larger simultaneous coverage. An example episode is shown in Fig. 6.

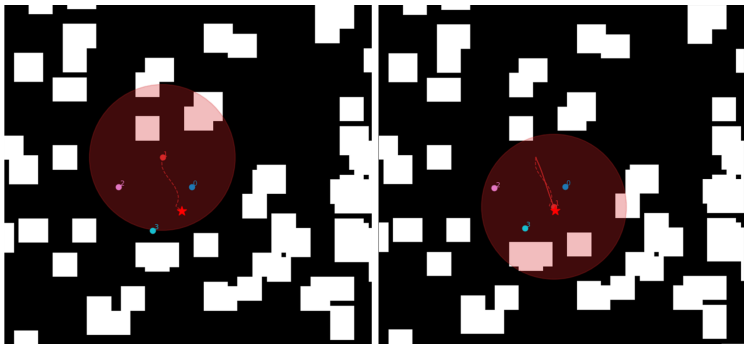


Figure 6: Example episode showing the lazy movement behavior. The leader robot (red) moves towards the goal, while the other robots (blue) remain stationary because they are always in communication range. The dotted line is the smoothed optimal A* path only for visualization purposes.

C Loss Functions

This section provides detailed loss function definitions in Eq. (4). Recall that the overall loss function is defined as

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{traj}} + \lambda_2 \mathcal{L}_{\text{wp}} + \lambda_3 \mathcal{L}_{\text{occ}} + \lambda_4 \mathcal{L}_{\text{sdf}}. \tag{7}$$

Besides the trajectory denoising loss $\mathcal{L}_{\text{traj}}$, the other three auxiliary losses are defined as follows. Note that all the three auxiliary losses can be calculated with the self-contained information from the training dataset, without the need of additional data collection or annotation, which we call multi-scale spatial-temporal self-supervision.

Waypoint Prediction Loss $\mathcal{L}_{\text{wp}}$. The waypoint head predicts K (we take $K = 5$) sparse future waypoints for each robot, defined as $\hat{L}^i = \{\hat{l}^i_1, \ldots, \hat{l}^i_K\}$, which are sampled from the expert trajectory in the training dataset at fixed distance intervals. If no additional waypoints can be sampled before reaching the goal, the remaining waypoints are set to the goal position. The waypoint loss is the mean squared error between predicted and target anchors:

$$\mathcal{L}_{\text{wp}} = \frac{1}{NK} \sum_{i=1}^{N} \sum_{r=1}^{K} \left\| \hat{l}^i_r - l^i_r \right\|_2^2. \tag{8}$$

This auxiliary target improves long-range planning by encouraging robot tokens to encode future progress beyond the short action chunk.

Occupancy Reconstruction Loss $\mathcal{L}_{\text{occ}}$. We define two local occupancy reconstruction heads that decode two occupancy patches from middle-layer latent vectors at two spatial scales ($1\times$ and $3\times$ in our implementation). The sizes of the occupancy patches are set to 7×7. Therefore, this task guides the model to learn the coarse spatial structure within a large 21×21 area around each robot, and the fine-grained structure within a small 7×7 area. The large patch is downsampled to the same 7×7 resolution as the small patch to reduce the capacity burden of the latent vectors. Given the ground-truth occupancy labels y^q_{hw} for each patch at scale $q \in \{\text{small}, \text{large}\}$, where $y^q_{hw} = 1$ if the cell is occupied by an obstacle, and $y^q_{hw} = 0$ otherwise, we supervise the occupancy reconstruction heads with binary cross entropy loss:

$$\mathcal{L}^q_{\text{occ}} = -\frac{1}{HW} \sum_{h,w} [y^q_{hw} \log \sigma(x^q_{hw}) + (1 - y^q_{hw}) \log(1 - \sigma(x^q_{hw}))], \tag{9}$$

where $\sigma(x) = 1/(1 + e^{-x})$. The combined occupancy loss is

$$\mathcal{L}_{\text{occ}} = \mathcal{L}^{\text{small}}_{\text{occ}} + \mathcal{L}^{\text{large}}_{\text{occ}}. \tag{10}$$

Signed Distance Loss. To encourage obstacle clearance, we use an SDF penalty on generated trajectories and discrete waypoints. We pre-construct the SDF map for each training environment,

where the value of each cell is the signed distance to the nearest obstacle (positive for free space and negative for occupied space). With the bilinear interpolation of the SDF map, we can obtain the signed distance for any point in the environment, with continuous gradients. As introduced in Sec. 4.2, the SDF loss is defined as

$$\mathcal{L}_{\mathrm{sdf}} = \frac{1}{N(T+K)} \sum_{i=1}^{N} \left(\lambda_{\mathrm{sdf}}^{\mathrm{traj}} \sum_{t=1}^{T} \left[\mathrm{ReLU} \left(d_{\mathrm{thres}} - d_{\mathrm{sdf}}(p_t^i) \right) \right]^2 + \sum_{r=1}^{K} \lambda_{\mathrm{sdf}}^{\mathrm{wp}} \left[\mathrm{ReLU} \left(d_{\mathrm{thres}} - d_{\mathrm{sdf}}(\tilde{l}_r^i) \right) \right]^2 \right), \tag{11}$$

where $d_{\mathrm{sdf}}(p)$ is the signed distance for point p obtained from the SDF map, and d_{thres} is a pre-defined distance threshold for obstacle clearance. We set both $\lambda_{\mathrm{sdf}}^{\mathrm{traj}}$ and $\lambda_{\mathrm{sdf}}^{\mathrm{wp}}$ to 1.0 in our implementation.

D Trajectory-Level Reinforcement Learning

Rollout dataset collection: We perform post-training reinforcement learning on top of the supervised pretrained Roken. Each RL iteration first constructs an on-policy rollout batch from the initial states sampled from the expert episode mid-states. Concretely, we sample an expert episode from the training set and randomly choose an intermediate timestep t when the remaining future trajectory is long enough for rollout evaluation. The prompt state is then formed as $c = (\mathcal{H}_{\text{map}}, \mathcal{P}_t, g, \{m_g^i\}_{i=1}^N)$, where $\mathcal{P}_t = \{p_t^i\}_{i=1}^N$ denotes the current team positions and $\{m_g^i\}_{i=1}^N$ is the binary goal mask. We also require the sampled multi-robot state to satisfy the same communication and safety constraints as defined in Sec. 3.2, so that RL starts from feasible team configurations.

For each prompt c_q, we generate a group of G candidate team trajectories from the current Roken mode, which can be denoted as a diffusion policy π_θ. All G candidates share the same conditioning input, but use independent diffusion noise:

$$\mathcal{T}_{q,j} \sim \pi_\theta(\cdot \mid c_q), \qquad j = 1, \ldots, G.$$

In our implementation, G is set to 32 for training rollouts. The diffusion model predicts normalized relative action sequences, which are converted into absolute trajectories by integrating from the current robot positions. These trajectories are then transformed back to the environment coordinate frame before reward computation. This produces a rollout batch of grouped candidates

$$\mathcal{B} = \{(c_q, \mathcal{T}_{q,1}, \ldots, \mathcal{T}_{q,G})\}_{q=1}^P,$$

where P is the number of prompts collected in one RL rollout.

Randomly sampling mid-states from the expert trajectories may result in an imbalanced rollout dataset, as most of the sampled states are relatively easy for the pretrained policy by directly navigating to the goal. To emphasize difficult states, we optionally mix in hard prompts with a pre-defined probability. The hard prompts are selected from mid-states where the pretrained policy previously generated invalid or low-quality candidates.

Reward function with privilege information: Each sampled team trajectory $\mathcal{T}_{q,j}$ is evaluated by a privileged reward function with additional simulator-side information, including the signed-distance field, geodesic distance to the goal, collision checks, communication connectivity checks, and the deviation from the optimal A* path. These privileged signals are used only during RL training and are not required at deployment. For a candidate team trajectory $\mathcal{T}$, the cost function is defined as

$$C(\mathcal{T}) = C_{\text{obs}}(\mathcal{T}) + C_{\text{inter}}(\mathcal{T}) + C_{\text{conn}}(\mathcal{T}) + C_{\text{goal}}(\mathcal{T}) + C_{\text{wp}}(\mathcal{T}),$$

where the first three terms capture the hard constraints of the task: C_{obs} penalizes obstacle collision by counting the number of collisions with soft margin; C_{inter} penalizes robot-robot collisions and soft minimum-distance violations using $\|p_t^i - p_t^j\|_2$; C_{conn} penalizes communication connectivity failures through $\mathcal{G}_t$ and $\lambda_2(\mathcal{L}_{\mathcal{G}_t})$, by counting the number of disconnected steps. If above hard constraints are violated, a large cost C_{max} is assigned to the trajectory to reduce its generation probability.

Besides that, we also inject privileged signals to encourage better planning performance: C_{goal} measures leader's final distance to the goal g based on geodesic distance; and C_{wp} penalizes deviation from the ground truth waypoints sampled from the optimal A* path. Note that C_{wp} is important for improving the long-horizon planning performance of the model, because it is insufficient to score candidate short action chunks to check whether it avoids local minima areas, while the long-horizon waypoints can provide larger horizon for candidate scoring. We also add a reach bonus once the leader robot reaches the goal to encourage task completion. The scalar reward is then obtained by clipping and rescaling the total cost:

$$r(\mathcal{T}) = -\frac{\min(C(\mathcal{T}), C_{\max})}{s}.$$

Here s controls the reward magnitude. Thus, safer trajectories with better leader goal progress receive larger rewards, while collision, disconnection, and obstacle violations receive lower rewards.

Training settings: We use a group-relative policy optimization objective following [29]. Since all G candidates in one group are sampled from the same prompt, their rewards can be compared directly without training a value function. For prompt q, let $r_{q,j} = r(\mathcal{T}_{q,j})$ be the reward of candidate j. We compute the normalized group-relative advantage

$$A_{q,j} = \frac{r_{q,j} - \mathrm{mean}_{l=1}^{G}(r_{q,l})}{\mathrm{std}_{l=1}^{G}(r_{q,l}) + \epsilon}.$$

If the reward standard deviation of a group is too small, all advantages in that group are set to zero, because the group does not provide a useful ranking signal.

During rollout generation, we store the stochastic DDPM reverse-process transitions for each sampled team trajectory using the current policy $\pi_{\theta_{\text{old}}}$. Specifically, for every denoising step, we save the noisy action chunk $\mathcal{T}_k$, the sampled next action chunk $\mathcal{T}_{k-1}$, the timestep k, and the behavior-policy log probability

$$\log \pi_{\theta_{\text{old}}}(\mathcal{T}_{k-1}^{q,j} \mid \mathcal{T}_k^{q,j}, c_q, k).$$

Each denoising step can be viewed as a decision-making process by sampling the next action chunk $\mathcal{T}_{k-1}$ from a multi-variate Gaussian distribution, denoted as

$$\mathcal{T}_{k-1}^{q,j} \sim \pi_{\theta_{\text{old}}}(\cdot \mid \mathcal{T}_k^{q,j}, c_q, k) = \mathcal{N}(\mu_{\theta_{\text{old}}}(\mathcal{T}_k^{q,j}, c_q, k), \sigma_k^2 I). \tag{12}$$

The variance σ_k^2 is determined by the noise schedule and is fixed during training, while the mean is determined by the predicted noise $\epsilon_{\theta_{\text{old}}}(\mathcal{T}_k^{q,j}, c_q, k)$ as

$$\mu_{\theta_{\text{old}}}(\mathcal{T}_k^{q,j}, c_q, k) = \frac{1}{\sqrt{\alpha_k}} \left(\mathcal{T}_k^{q,j} - \frac{1-\alpha_k}{\sqrt{1-\bar{\alpha}_k}} \epsilon_{\theta_{\text{old}}}(\mathcal{T}_k^{q,j}, c_q, k) \right), \tag{13}$$

where α_k and $\bar{\alpha}_k$ are the noise schedule parameters.

During policy optimization, we recompute the log probability of the same denoising transition under the current policy π_θ and form the likelihood ratio

$$\rho_{k,q,j} = \exp\left(\log \pi_\theta(\mathcal{T}_{k-1}^{q,j} \mid \mathcal{T}_k^{q,j}, c_q, k) - \log \pi_{\theta_{\text{old}}}(\mathcal{T}_{k-1}^{q,j} \mid \mathcal{T}_k^{q,j}, c_q, k) \right).$$

The policy is updated with a clipped PPO-style objective:

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E}_{k,q,j}\left[\min\left(\rho_{k,q,j} A_{q,j}, \mathrm{clip}(\rho_{k,q,j}, 1-\epsilon_{\text{clip}}, 1+\epsilon_{\text{clip}}) A_{q,j} \right) \right].$$

To prevent the RL update from destroying the supervised diffusion prior, we keep a frozen reference model π_{ref}, initialized from the supervised checkpoint of Roken, and add an analytic Gaussian KL penalty at every denoising transition:

$$\mathcal{L} = \mathcal{L}_{\text{GRPO}} + \beta \mathbb{E}_{k,q,j}\left[D_{\text{KL}}\left(\pi_\theta(\cdot \mid \mathcal{T}_k^{q,j}, c_q, k) \,\|\, \pi_{\text{ref}}(\cdot \mid \mathcal{T}_k^{q,j}, c_q, k) \right) \right].$$

After collecting one rollout buffer, we perform minibatch updates for a small number of epochs, then discard the buffer and sample a new on-policy batch from the updated diffusion policy. This procedure keeps the RL data on-policy while using the frozen reference model to maintain stable multi-robot trajectory generation following the supervised prior.

RL post-training hyperparameters:

Table 5: RL post-training hyperparameters.

Hyperparameter	Value
Group size G	32
PPO clip threshold ϵ_{clip}	0.1
KL coefficient β	0.2
Number of rollout iterations	200
Rollout buffer size	500×32
Learning rate	1e-6
AdamW weight decay	0.01
Gradient clipping	0.5

Table 6: Reward cost function parameters.

Cost term	Segment weight
Obstacle collision C_{obs}	30.0
Inter-robot collision C_{inter}	30.0
Communication disconnection C_{conn}	30.0
Leader goal progress C_{goal}	30.0
Leader waypoint deviation C_{wp}	50.0
Reach bonus	50.0
Cost clipping $C_{\max}$	10000
Reward scale s	20

E Supplementary Experiments Results

E.1 Ablations on Relative / Absolute Trajectory Representation

We investigate the effect of different trajectory representations on the performance of Roken, including the trajectory represented with normalized absolute positions, and a relative representation with relative motion between consecutive time steps.

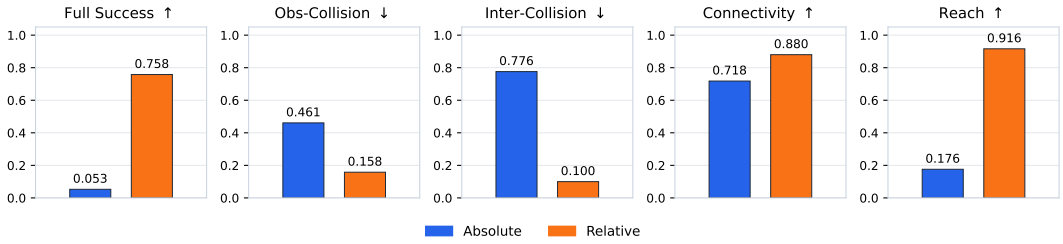


Figure 7: Ablation on absolute and relative trajectory representations. The results are evaluated over 500 episodes in 500 unseen maps.

As shown in Fig. 7, Roken with absolute trajectory representation fails to learn meaningful multi-robot coordination distributions. The full success ratio is only 5.3%, compared with 75.8% for the relative representation. Although we have normalized the absolute positions to a fixed range, the model still struggles to learn. The reason is that the absolute trajectory representation buries the underlying coordination patterns across different samples by applying different initial position offsets. For example, two identical episodes with a slight shift of the map and trajectories will be treated as two distinct samples with the absolute representation, while they share exactly the same relative movements. Therefore, we use the relative trajectory representation in all our experiments to facilitate the learning of multi-robot coordination patterns.

Note that in all our experiments, we use the normalized absolute position for robot token position encoding, rather than the relative position representation w.r.t. the leader robot. With this setting, we found that each token can successfully cross-attend to the relevant map information and other

robots' states to generate coordinated trajectories. It further simplifies the application of the Roken model without the need of additional relative position transformation.

E.2 Implementations of Masked Training and Clustered Rollout

The clustered rollout is supported by the masked training of Roken, where the model predicts the trajectories of a subset of robots conditioned on the previously generated trajectories of the other robots. With this, we can scale the pre-trained Roken to even larger team sizes by dividing the team into overlapped clusters and performing rollout for each cluster sequentially. The trajectories of the overlapped robots are kept fixed as conditions.

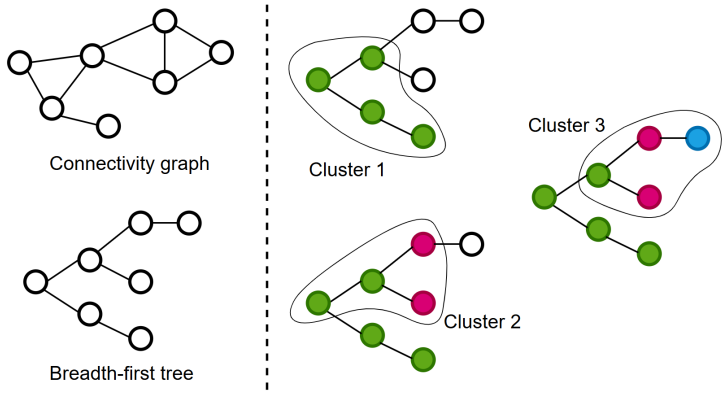


Figure 8: Example of clustered rollout. After Breadth-first tree construction, we cluster the robots into overlapping groups, which can then be generated sequentially. With masked training of Roken and clustered rollout, we can scale the pre-trained Roken only on the four robot dataset to 7 robots without additional training.

Overlapped clustering: As shown in Fig. 8, the overlapped clustering of the team includes three steps: (1) communication graph construction, as introduced in Sec. 3.1; (2) BFS tree construction with the leader robot as the root; and (3) cluster construction from the BFS tree. In cluster construction, the first cluster is rooted at the leader robot, and includes its children in the BFS tree until reaching the cluster size limit (which we set as 4). Starting from the second cluster, we iteratively select a robot that has been generated in the previous clusters as the root, and construct a new cluster with its children until reaching the cluster size limit. If the unvisited children are not enough to fill the cluster, we will further expand the cluster with the siblings of newly added children, and further the siblings or ancestors of the root robot until reaching the cluster size limit. There are three possible conditional generation cases: (1) one parent condition robot token with three unvisited children tokens; (2) two conditional robot tokens with two unvisited children tokens; and (3) three conditional robot tokens with one unvisited child token. We bias the case (3) by adding visited siblings or ancestors to fill the cluster, which can provide more informative conditions for the generation of the unvisited child token, at the cost of more clusters to cover the full team.

Clustered rollout can scale the pre-trained Roken to larger team sizes without additional training, at the cost of more generation steps and thus more compounding errors. The scalability performance of the clustered rollout is shown in Fig. 9 in the Appendix, where the pre-trained $\text{Roken}_{\text{four}}^{\text{masked}}$ can be applied to even ten robots with a full success ratio of 68.4%, in contrast to the 10.5% success ratio of $\text{Roken}_{\text{four}}$ without masked training.

E.3 Details of Scalability Experiments

Note that the scalability performance of $\text{Roken}_{\text{four}}^{\text{masked}}$ is better than the results in Fig. 4 in the main paper, after we fix a bug in previous code. The model $\text{Roken}_{\text{four}}^{\text{masked}}$ remains unchanged as in the main paper.

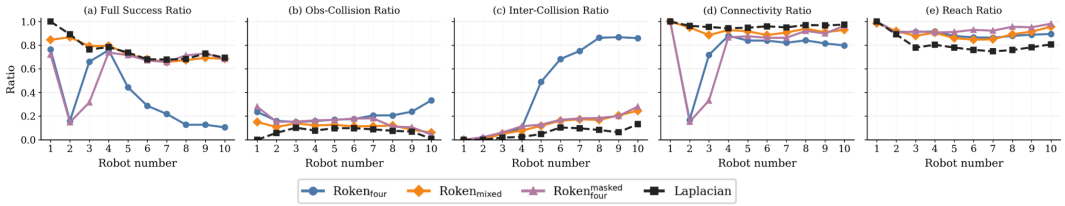


Figure 9: Scalability of Roken to larger team sizes.

The updated results are shown in Fig. 9. The scalability performance of $\text{Roken}_{\text{mixed}}$, $\text{Roken}_{\text{four}}^{\text{masked}}$, and the baseline Laplacian method [8] are similar, especially when the team size is large. However, we have to highlight that Laplacian uses the global optimal A* path for the leader robot as the reference trajectory, which is not available for Roken.

Without masked training, the $\text{Roken}_{\text{four}}$ model only scales to 3 or 5 robots with moderate success ratios, but fails to scale to more robots. In contrast, the masked training of $\text{Roken}_{\text{four}}^{\text{masked}}$ provides conditional planning capability to the model, thereby can scale to much larger team sizes with clustered rollout, as shown in Fig. 9. The performance of clustered rollout is even similar to the direct rollout of $\text{Roken}_{\text{mixed}}$ trained with mixed team sizes. Therefore, we claim that the masked training and clustered rollout provides a scalable way to either directly apply the pre-trained Roken to larger team sizes without additional training, or to generate expert trajectories of larger teams for further pre-training of Roken.

E.4 Ablations on Replanning Frequency

In both our simulation rollout and real-world deployment, we adopt a receding horizon control scheme where Roken generates a trajectory chunk with 16 steps at every control cycle, but only executes the first two steps before re-planning with the updated multi-robot state and occupancy map. Here we provide results of Roken with different replanning frequencies by changing the number of executed steps before the next re-planning, as shown in Fig. 10. The full success ratio decreases as the replanning frequency decreases, which indicates that more frequent replanning can help to correct errors in the predicted trajectory with more updated states. Therefore, we set the replanning frequency to 2 steps in all our evaluations to get the best performance.

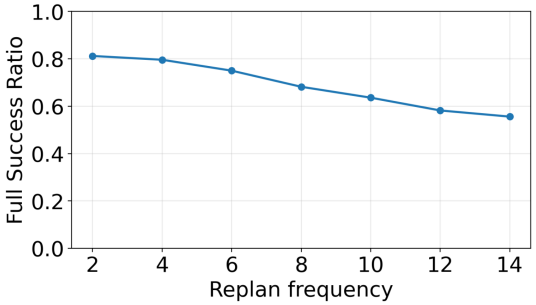


Figure 10: Ablation on replanning frequency. The model $\text{Roken}_{\text{four}}$ is rollout in 500 episodes with 500 unseen maps. The robot number is fixed to four.

E.5 Sim-to-Real Gap and Real-World Deployment

To deploy the trained Roken to real-world multi-robot systems, we need to consider the sim-to-real gap, especially how to apply Roken (trained with fixed parameters) to different communication and safety requirements.

- **Obstacle avoidance with different robot sizes:** This constraint can be addressed by adjusting the expansion radius of the obstacles in the occupancy map.

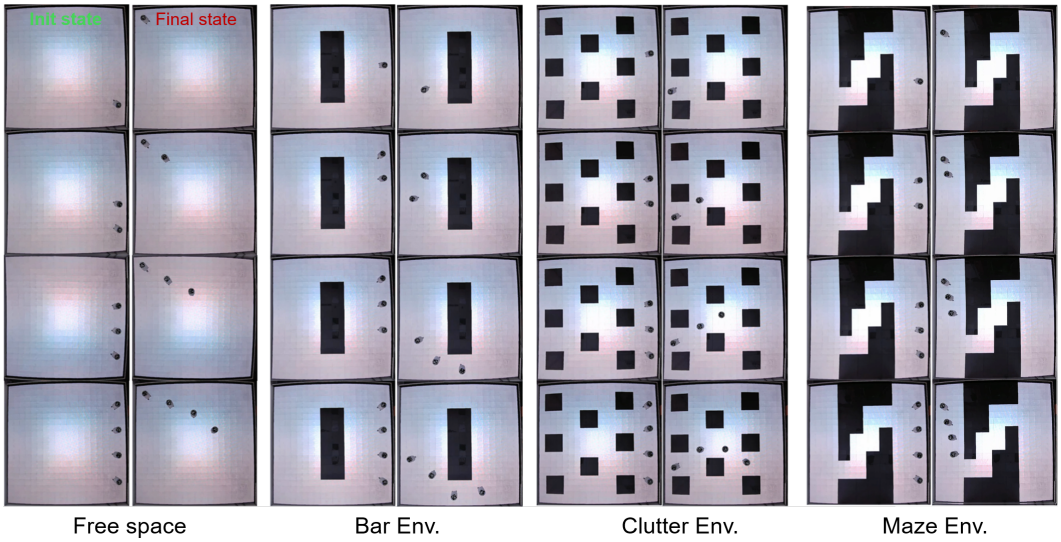


Figure 11: Real-world deployment of Roken. For each environment, the left column shows the initial state and the right column shows the final state after navigation. One Roken model can successfully control one to four robots to navigate to the goal while maintaining connectivity and avoiding collisions.

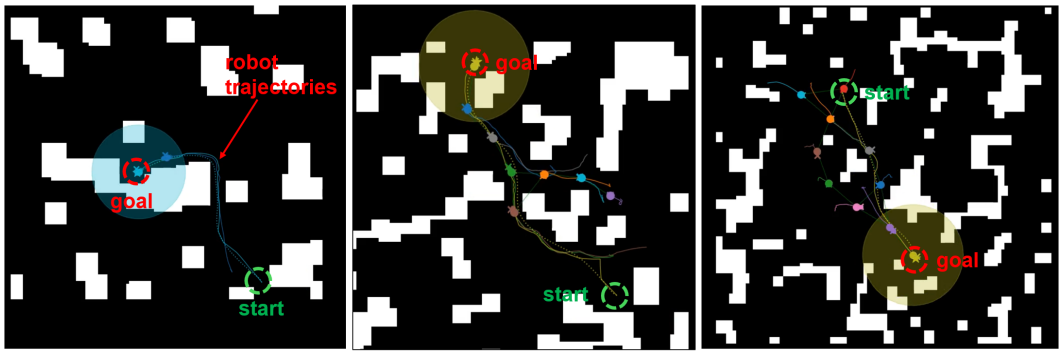
- **Communication maintenance with different radius:** This constraint can be addressed by scaling the local occupancy maps with different resolutions. The pre-defined communication range is 15 pixels in the input occupancy map where 1 pixel represents 1 meter. Therefore, shrinking the original $\mathcal{H}_{\text{map}}$ to a smaller size will result in a larger communication range, while expanding the original $\mathcal{H}_{\text{map}}$ will result in a smaller communication range (as one pixel now represents less than 1 meter).
- **Inter-robot collision avoidance:** The current Roken cannot directly adapt to different inter-robot collision distances. In our real-world deployment, this limitation is not a problem because the *lazy movement* behavior of the trained Roken policy encourages robots to stay still when possible to reduce the efforts for connectivity maintenance. Therefore, the robots tend to form a chain-like formation in long-term navigation tasks. To strictly impose a different inter-robot collision distance, a safety shield module is needed to perform online trajectory correction when the predicted trajectory violates the inter-robot collision constraint.

For real-world deployment, we use four WheelTec robots[2] with omnidirectional mecanum wheels. The localization of each robot is obtained by the AprilTag-based system[3], and the occupancy map provided as a prior. The robots track the generated trajectories with a simple PI controller with a control frequency of 100Hz. The deployment results of one to four robots are shown in Fig. 11. The results are also included in the supplementary video.

E.6 More Rollout Results

[2]https://wheeltec.net/

[3]https://github.com/AprilRobotics/apriltag



(a) 2 robots (b) 8 robots (c) 10 robots

Figure 12: Rollout episodes of $\text{Roken}_{\text{mixed}}$ for 2, 8, and 10 robots.

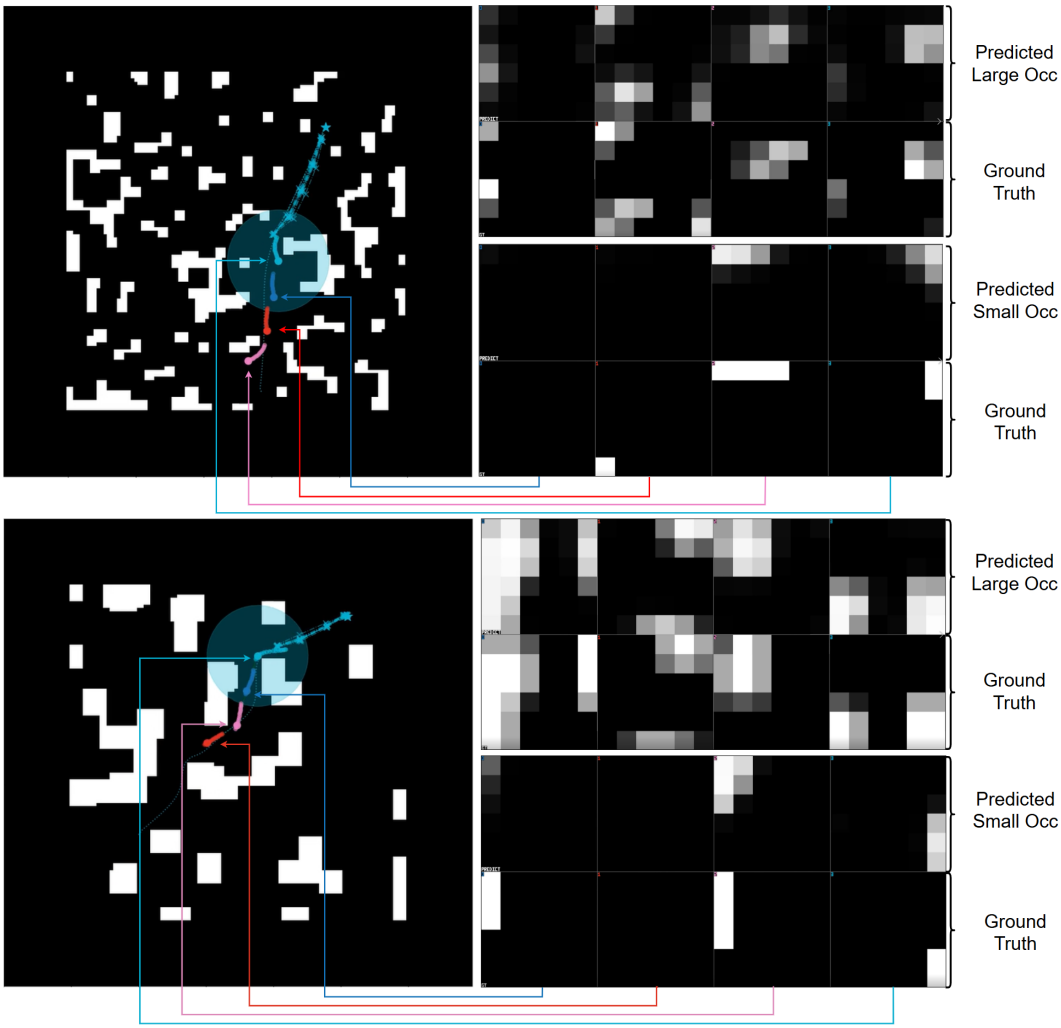


Figure 13: Examples of both large and small local occupancy reconstruction results. Each row corresponds to one robot. The predicted occupancy patches demonstrate that in Roken, robots can successfully attend to multiple discrete map tokens, aggregate them, and reconstruct their surrounding local occupancy pattern.

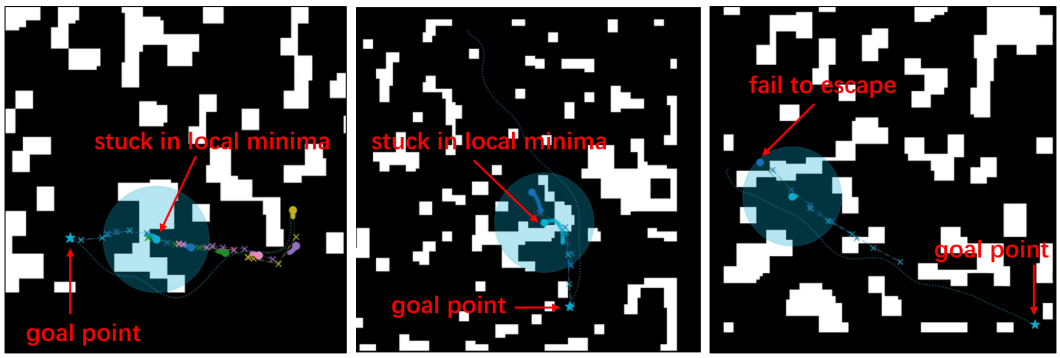


Figure 14: Example failure cases of Roken. Roken fails to generate valid trajectories that escape from the local minima areas, because the training dataset does not include expert demonstrations in such scenarios. This can be further improved by dataset enhancement and RL post-training.